

HIGH-LEVEL SYNTHESIS

Geometric Algorithm Scheduling

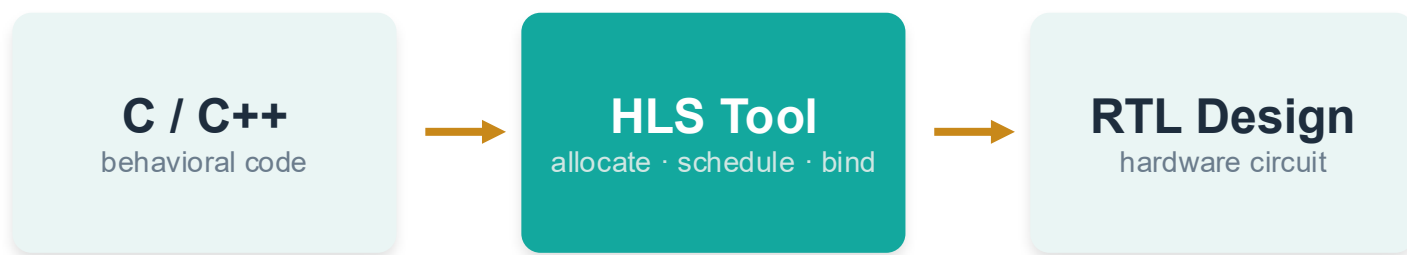
Turning the hardware scheduling problem into geometry — the GEM approach

Md. Tawhidur Rahman Tuhin

Electronic Engineering · Hochschule Hamm-Lippstadt, Germany

What is High-Level Synthesis?

HLS automatically converts a C / C++ program into a hardware circuit (RTL), so designers no longer have to build hardware by hand — a slow, error-prone job.



Three coupled stages

Allocation — how many adders, multipliers, registers to use.

Scheduling — assign each operation to a clock cycle (C-step).

Binding — map each operation onto a concrete hardware unit.

Scheduling is the hard part

It decides how many clock cycles the circuit needs (its latency) and how busy each resource is. Good scheduling directly drives both speed and chip area.

THE PROBLEM

Why scheduling is hard

With unlimited hardware, scheduling is trivial — just run everything as soon as its inputs are ready. The challenge is limited resources.

NP-hard

The resource-constrained version is NP-hard — e.g. only 2 multipliers for 6 multiplications means operations must be spread across cycles without overloading anything, while keeping the schedule short.

Traditional approaches

ASAP / ALAP Simple earliest / latest placement — gives bounds only.

List Scheduling Fast heuristic, but no quality guarantee.

Force-Directed Higher quality, but slows to $O(|V|^2)$ on dense graphs.

Integer Linear Prog. (ILP) Optimal — but can stall for minutes.

THE CORE IDEA

GEM: turn scheduling into geometry

Instead of reasoning about operations and time slots directly, GEM plots every operation as a point on a 2-D plane and solves a clean geometry problem.

$$\pi(v) = (\alpha(v) , \beta(v))$$

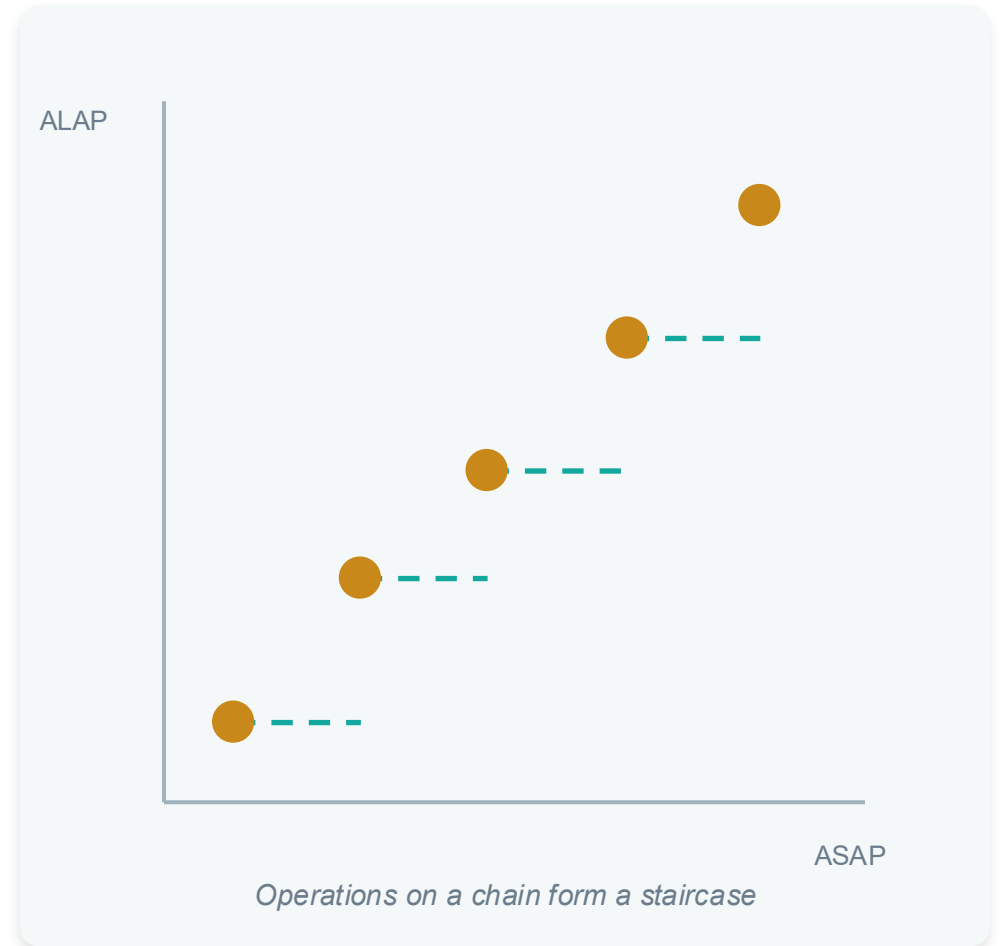
x = ASAP time

the earliest cycle an operation can run

y = ALAP time

the latest cycle it can run without delaying the design

Operations on the same dependency chain form a staircase — each point sits up and to the right of the previous one. A tight staircase means little flexibility.



Two ideas that make it work

1

Point dominance

A point with higher ASAP and ALAP dominates another. Dominated operations have less wiggle room, so GEM gives them priority — focusing effort where it matters most.

2

Non-crossing matching

Each operation is matched to a clock cycle so the order is preserved (precedence). Finding the best such matching is solved efficiently in $O(N \log N)$ with a sweep.

GEM in four phases

1

Compute ASAP & ALAP

Two passes over the graph give every operation its mobility window — turning each into a 2-D point.

2

Decompose into paths

Greedily break the graph into the fewest straight dependency chains; tight operations stay together.

3

Match each path

Run non-crossing matching per chain — provably optimal for a single chain.

4

Merge & resolve

Combine chains; where a cycle is overloaded, push the most flexible operation later and re-match.

WORKED EXAMPLE

The HAL benchmark

A standard test from a numerical-integration loop: 6 multiplications, 2 subtractions, 1 addition, 1 comparison — with a budget of 2 multipliers and 1 ALU.

What GEM does

- Splits the graph into 4 dependency chains.
- The chain $m1 \rightarrow m2 \rightarrow m3$ has zero mobility — it's the critical path.
- Cycle 1 over-uses multipliers, so $m6$ (most flexible) is pushed to cycle 2.
- Cycle 2 over-uses the ALU, so $c1$ is pushed to cycle 3.
- Two quick iterations converge on the answer.

4

clock cycles

GEM recovers the exact optimal schedule found by exhaustive search — using just 2 multipliers and 1 ALU.

Strengths & limitations

Advantages

- **Fast** — Runs in milliseconds where ILP can stall for minutes.
- **Optimal on chains** — Provably best schedule for any dependency chain.
- **Loops & branches free** — Path decomposition handles them with no extra cost.
- **Easy to implement** — Core primitive is ~30 lines of C++ using only `std::sort`.

Limitations

- **Guarantee stops at the chain** — Full-graph result can drift 1–2 cycles from optimum.
- **Greedy conflict resolution** — Phase-4 push rule is local, with no global bound.
- **No loop pipelining** — Schedules one iteration only — key for modern throughput.
- **Limited industry uptake** — Commercial tools favour tuned ILP and list scheduling.